

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



3일차

04 CAN 통신 - D3-G A72

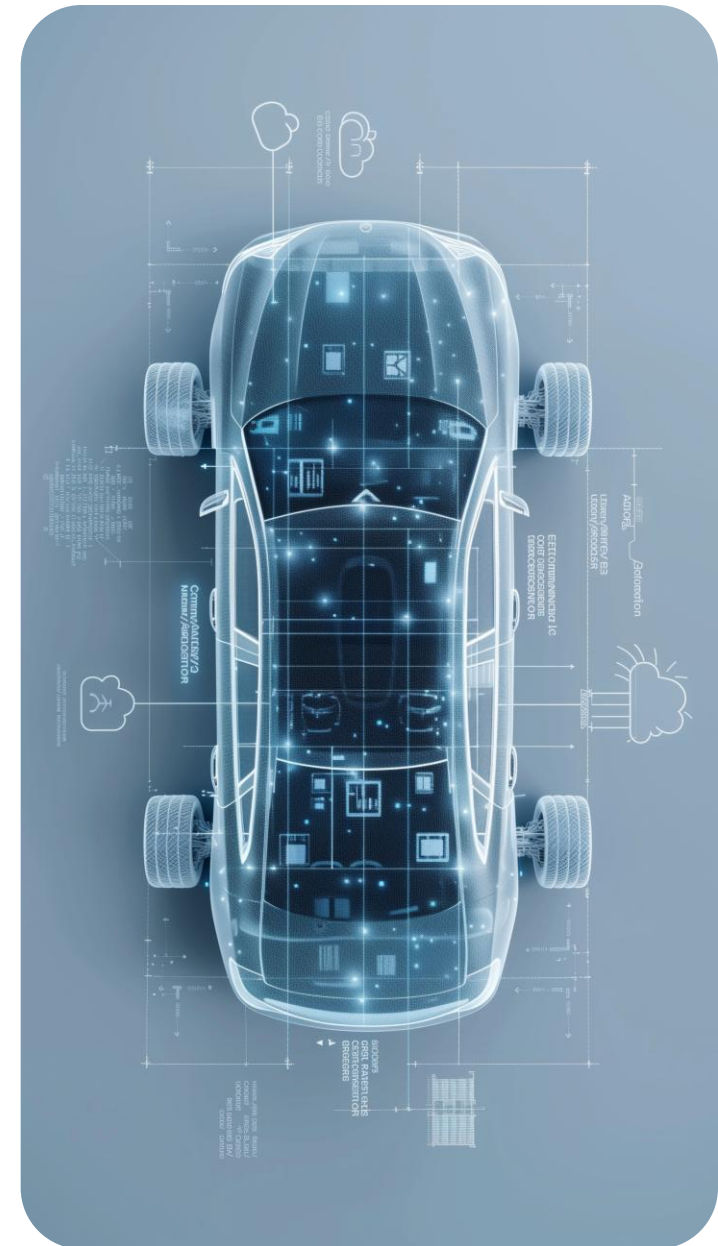
Telechips TOPST



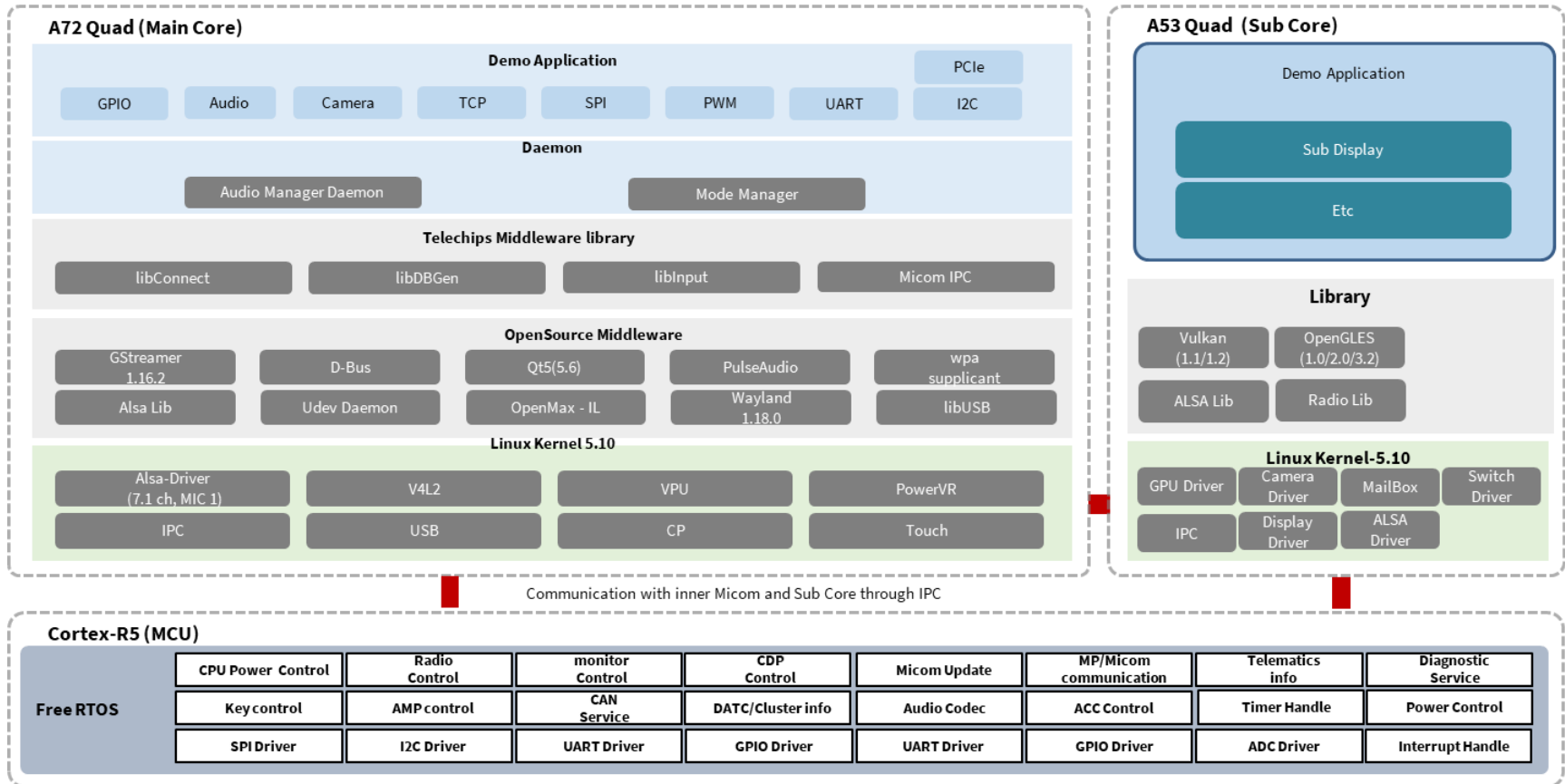
목차

Table of Contents

D3-G CAN 통신 구조	01
D3-G A72 core : IPC process	02
D3-G A72 core : IPC Example setting	03



D3-G Software Architecture



SW Components may differ from the officially distributed SDK

CAN support in D3-G



SW Components may differ from the officially distributed SDK

D3-G IPC 이해하기



D3-G chip : tcc8050

- IPC : Inter Processor Communication 의 약자로, CPU와 MCU 간의 통신을 지원하기 위한 통신 구조

Q) CAN 드라이버를 CPU에서 직접 안 쓰나요?

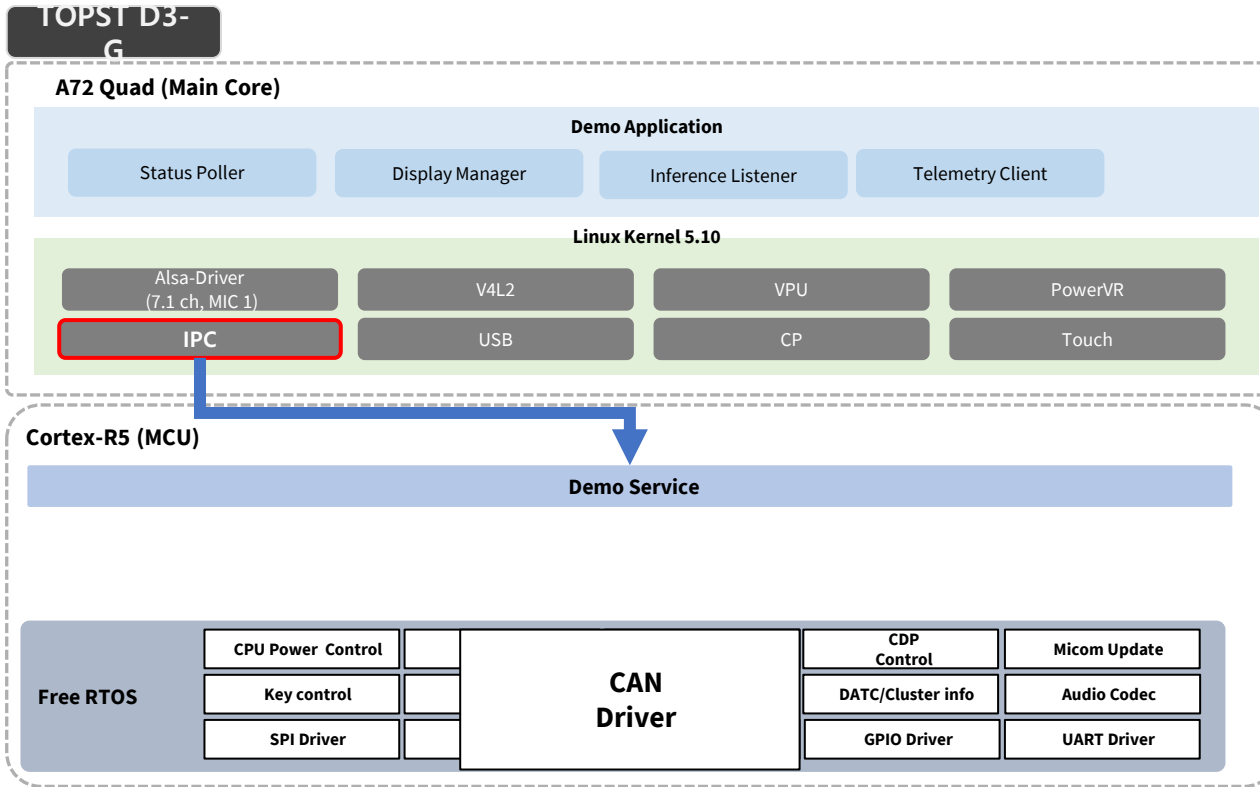
➔ A72는 기존 IVI를 담당하는 코어, A72가 CAN 까지 관리한다면, 코어가 예기치 못한 오류로 죽을때, 차량 제어 영역이 매우 critical해짐.

Q) 그래서 CPU가 CAN을 사용하기 위해선?

➔ CPU > IPC > MCU > CAN 드라이버 > 차량 네트워크

쉽게 설명하여, IPC 는 CPU가 MCU에게 "이 메시지를 좀 보내줘" 라고 전달하는 우체부 역할

D3-G CAN 실습 Preview



*IPC : Inter Processor Communication

1. D3-G to MCU IPC를 이용해 상태 전달
 - CAN Message와 유사한 별도의 User-defined protocol을 사용
2. MCU에서 user-defined protocol을 수신하고 해석
3. Micom에서 CAN 통신 포트 설정 (init, open 등)
4. 전송할 항목을 CAN 통신 프로토콜로 전송

본 과정의 실습은 CAN 통신 실습을 위해 TOPST D3-G 보드를 사용하며 코어당 수행하는 기능별로 설명

- A72 core : IPC Task 수행
- R5 core : CAN Controller 수행

A72 IPC Task 이해

- IPC_Example.py의 전체 동작 구조는 아래와 같음
1. A72 쪽 사용자 입력 (IPC_Example.py)
 - argparse로 모드와 데이터를 입력 받음 (snd /rev / sndrecv)
 - snd : send
 - rev : receive
 - sndrecv : send and receive
 2. 패킷 구성 및 송신 요청
 - IPC_SendPacketWithIPCHheader() 호출
 - 데이터 -> 헤더 + 명령 + CRC 구조로 패킹
 - /dev/tcc_ipc_micom 파일로 전송
 3. R5 쪽 수신 및 응답
 - R5 가 패킷을 수신 -> CAN 드라이버로 전달 (rev 모드에서는 응답 패킷 수신)



A72 IPC Task 이해

• IPC_Example.py 내 main()

```
def main():
    parser = argparse.ArgumentParser(description="IPC Sender/Receiver")
    parser.add_argument("mode", choices=["snd", "rev", "sndrecv"], help="'snd': send, 'rev': receive, 'sndrecv': send then receive once")
    parser.add_argument("--file_path", default="sndfile", help="File path for IPC communication") # sndfile = open("/dev/tcc_ipc_micom", 'wb')
    parser.add_argument("--channel", type=int, default=1, help="can channel bit flag: b111")
    parser.add_argument("--txOnly", type=int, default=0, help="tx only channel bit flag: b111 (dec: 1, 2, 4) ")
    parser.add_argument("--canID", type=parse_hex16, default=0, help="CAN ID max 16-bit: e.g. 123 or 0x1234")
    parser.add_argument("--sndDataHex", type=str, help="Value for sndData as a hex string, e.g., '12345678' (default: string)")
    parser.add_argument("--sndData", type=str, default="12345678", help="Default data if no sndDataHex provided")

    ...

    sndDataLength = len(sndData)
    channel_str = ", ".join(parse_channels(channel_bitmask))
    txonly_str = ", ".join(parse_channels(tx_only_channel_bitmask))
    print(f"channel_bitmask: 0x{channel_bitmask:02X} (channels: {channel_str or 'None'})")
    print(f"tx_only_channel_bitmask: 0x{tx_only_channel_bitmask:02X} (channels: {txonly_str or 'None'})")
    print(f"canID: 0x{canID:04X}")
    print(f"sndData: {sndData}")
    print(f"sndDataLength: {sndDataLength}")
    ret = IPC_SendPacketWithIPCHheader(args.file_path, channel_bitmask, tx_only_channel_bitmask, canID, sndData)
```

- --file_path 는 /dev/tcc_ipc_micom 을 기본으로 사용
- IPC_SendPacketwithIPCHheader 함수를 통해, canID, sndData 전송에 중점
- 전송모드 snd, recv, sndrecv 명시 가능하나,
- --channel : CAN 통신 채널을 의미하며 1, 2, 4는 각각 0번, 1번, 2번 채널을 의미
- --canID : CAN 노드의 Identifier 를 의미
- --sndDataHex : 전송하고자 하는 전체 data를 의미하며 HEX 값으로 2자리씩 붙여서 명시한다 (ex : 0x123456 를 전송할때 '123456' 로 명시)

A72 IPC Task 이해

- IPC_SendPacketWithIPCHheader()의 구조는 아래와 같음
- function definition

IPC_SendPacketWithIPCHheader(sndFile, channel_bitmask, tx_only_channel_bitmask, canID, pucData)

- function flow

1. 빈 패킷 생성 (array)
2. 헤더 작성 (SYNC, START1, START2)
3. 길이 설정 (LENGTH)
4. CM3 추가 (Channel, txOnly, CAN ID)
5. 데이터 삽입(pucData)
6. CRC 계산 후 붙이기
7. bytes로 변환
8. /dev/tcc_ipc_micom에 write()

```
packet_send = array.array('B', [0] * IPC_MAX_PACKET_SIZE)
...
cnt = 0
for cnt in range(pucData_len):
    packet_send[IPC_PACKET_PREPARE_SIZE + uiCmd3_size + cnt] = pucData[cnt]
...
packet_bytes = bytes(packet_send[:total_size])
...
while True:
    try:
        sndFile.write(packet_bytes)
        now = time.time() * 1000
        sndFile.flush() #
    ...
```

A72 IPC 세팅 & 실행

1. 1일차에 FWDN 한 ubuntu d3-g를 mobaXTerm으로 연결한다.
2. 아래와 같은 커맨드를 입력해서, tcc_ipc_micom 가 설정되어 있는지 확인한다.

```
topst@TOPST:~$ ls -al /dev/tcc_ipc*
crw----- 1 root root 244, 0 Jun  4 14:17 /dev/tcc_ipc_ap
crw----- 1 root root 245, 0 Jun  4 14:17 /dev/tcc_ipc_micom
topst@TOPST:~$
```

3. 아래와 같이 실습용 예제 코드 확인

```
$ cd ../home/topst/Fabless_Education_blink/Step_0
```

```
$ ls
```

- IPC_Example.py 사용예정
 - IPC_Example.py - MCU로 메시지 송수신하는 IPC 코드 예제
 - IPC_Library.py - MCU로 메시지 송수신하는 IPC 라이브러리

```
topst@TOPST:~/Education/d3-g app$ tree
.
├── Data_parsing_CAN.py
├── Data_parsing.py
├── Data_parsing_QT.py
├── IPC_Example.py
├── Library
│   └── IPC_Library.py
```

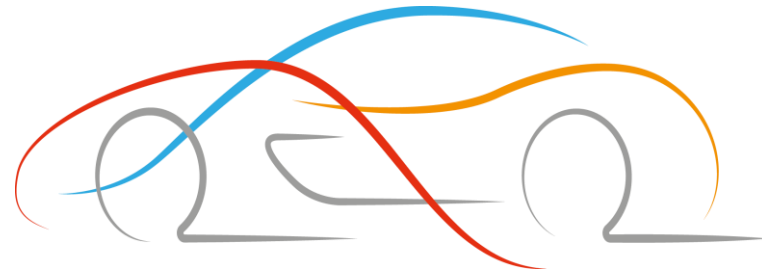
A72 IPC 세팅 & 실행

• IPC 전송 예제 실행

- 시리얼 통신으로 연결된 Putty 로그를 통해 A72에서 전송한 메시지를 아래 커맨드로 확인할 수 있다.
- `$ sudo python3 ./IPC_Example.py snd -sndData 1122`

```
topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd --sndData 1122
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0000
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0000
sndData: b'1122'
sndDataHex: 31313232
sndDataLength: 4
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$
```

- CAN Channel , ID 는 default 값 사용
- Data 1122를 송신
- 출력되는 결과 : byte 1122 정상 출력완료
- 송신 데이터 길이 : 4 정상 출력완료



Thank You